DAY 7 DSA

Complete Lecture-Based Study Notes

Hashing & Java HashMap • Binary Search Trees • Tree Height and Deletion • Iterative Traversals • Breadth-First Search

Prepared from the Day 7 lecture transcript. Additional code snippets and dry runs are clearly presented as learning examples to help explain the lecture concepts.

How to use these notes

Start with the concept, trace the example by hand, and then type the code yourself. For tree algorithms, always identify the base case, what happens at the current node, and the order in which subtrees are processed.

Lecture scope: The transcript revisits open addressing and Java HashMap internals, then focuses on binary-tree height, deleting all nodes, non-recursive preorder traversal using a stack, breadth-first/level-order traversal, DFS vs BFS, using parent pointers, and deleting a specific node from a tree. The lecturer indicates that some tree/sorting topics continue in a later session.

1. Hashing: open addressing search

In open addressing, all entries live inside the hash table's array. When a collision occurs, a probing rule finds another slot. A search must follow the same probe sequence that insertion used.

How do we stop searching when a key is absent?

• If the computed slot is empty (for the standard insertion/search rules), the key was not inserted along this probe sequence, so search can report “not found.”

• If the slot contains the requested key, return its value.

• If the slot contains another key, calculate the next probe slot and continue.

• To guard against a full table or unusual probing cycle, also stop after checking at most the table capacity number of slots.

Important implementation detail: deletion in open addressing often needs a special “deleted/tombstone” marker. Treating a deleted slot as simply empty can break searches for keys that were placed later in the same probe sequence.

Simple pseudocode

```
index = hash(key)
checks = 0
while checks < table.length:
if table[index] is EMPTY:
return NOT_FOUND
if table[index].key equals key:
return table[index].value
index = nextProbe(index, checks)
checks = checks + 1
return NOT_FOUND
```

The lecture describes using null references or a sentinel/flag to mark unused slots. If using null, check for null before accessing a slot's key to avoid a NullPointerException.

2. Java HashMap: what happens internally?

The lecture explains HashMap as a hash-table implementation using buckets (often called bucketing or chaining for collision handling). A key is used to determine the bucket; an entry holding the key and value is stored in that bucket.

Example use cases

• Map a roll number to a Student object: HashMap.

• Map a person's name to a mobile number: HashMap.

• Map a Student object to a Marksheet object: HashMap.

Conceptual put(key, value) flow

```
1. Call hashCode() on the key.
2. Transform/spread the hash and calculate a bucket index.
3. Go to that bucket.
4. If no matching key exists, add an entry.
5. If an equal key already exists, replace/update its value.
```

Conceptual get(key) flow

```
1. Call hashCode() on the lookup key.
2. Find the bucket index.
3. Search entries in that bucket.
4. Use equals() to identify the exact matching key.
5. Return its value, or null if no mapping is found.
```

The exact internal bucket-index calculation and collision structures depend on the Java version and implementation. The transcript uses the familiar default-capacity/load-factor illustration (capacity 16, load factor 0.75) to explain resizing; treat these as standard defaults for common Java HashMap implementations, not as universal rules for every map implementation.

3. hashCode() and equals(): the contract

The two methods solve different parts of lookup. hashCode() helps locate a bucket efficiently. equals() checks whether a candidate key in that bucket is logically the same key.

Rule	Meaning
If a.equals(b) is true	a.hashCode() must equal b.hashCode().
If hash codes are equal	Objects do not have to be equal; collisions are allowed.
If objects are unequal	Their hash codes may still be the same.
Consistency	If fields used by equals() have not changed, repeated hashCode() calls should return the same value during the execut
Good performance	Different objects should, where practical, spread across hash codes/buckets to reduce collisions.

Why both methods are needed

Imagine two different keys land in the same bucket. The hash code identifies the bucket, but it cannot necessarily identify which entry is the right one. equals() distinguishes the keys inside that bucket.

A common mistake

Do not implement equals() using one identity field while hashCode() is based on an unrelated field. The methods must be consistent with the same logical definition of equality.

```
// Illustrative Student identity based on rollNumber
@Override
public int hashCode() {
return Integer.hashCode(rollNumber);
}

@Override
public boolean equals(Object obj) {
if (this == obj) return true;
if (!(obj instanceof Student)) return false;
Student other = (Student) obj;
return this.rollNumber == other.rollNumber;
}
```

This is an example only. If your class defines students as equal by more than roll number, both methods should use those same fields.

Interview true/false

• Two unequal objects may have the same hash code — TRUE.

• Two equal objects may have different hash codes — FALSE; that violates the contract.

• A hash function that returns the same number for every object can technically be valid, but it causes many collisions and poor performance.

4. Binary-tree API design and encapsulation

The lecture recommends exposing readable operations such as add(), preorder(), or height() instead of making callers manipulate internal nodes directly. A wrapper method hides implementation details and reduces the chance that outside code will misuse the tree's internal structure.

```
class BinaryTree {
private Node root;

public void preorder() {
preorder(root); // wrapper
}

private void preorder(Node node) {
if (node == null) return;
System.out.print(node.data + " ");
preorder(node.left);
preorder(node.right);
}
}
```

The public method gives the caller a simple interface; the private helper handles recursion and node references.

5. Deleting all nodes in a tree

The lecture explains two approaches. Setting root = null makes the tree stop referencing the root and, if no other references to its nodes exist, the nodes become eligible for garbage collection. A recursive postorder cleanup can explicitly clear child references first.

Why postorder?

Postorder processes the left subtree, then the right subtree, then the current node. Children are handled before their parent, which is the natural order when clearing references.

```
private void deleteAll(Node node) {
if (node == null) return;

deleteAll(node.left); // clear left subtree
deleteAll(node.right); // clear right subtree

node.left = null;
node.right = null;
}

public void clear() {
deleteAll(root);
root = null;
}
```

Java does not provide a manual delete operator for ordinary objects. The code removes references; garbage collection reclaims memory when objects are no longer reachable.

6. Height of a binary tree

In the convention used by the lecture, an empty/null tree has height -1, a leaf has height 0, and a tree's height is the maximum number of edges from the root to a leaf.

```
height(node):
if node == null:
return -1

leftHeight = height(node.left)
rightHeight = height(node.right)

return max(leftHeight, rightHeight) + 1
```

Dry run

Node	Left height	Right height	Calculation	Height
Leaf 20	-1	-1	max(-1,-1)+1	0
Node 10	0	0	max(0,0)+1	1
Node 40	-1	-1	max(-1,-1)+1	0
Node 30	1	0	max(1,0)+1	2
Root 50	2	2	max(2,2)+1	3

The public height() method can call a private recursive helper with root. After clear(), height() returns -1 under this convention.

7. Recursive traversals recap

Traversal	Order	Memory aid
Preorder	Node → Left → Right	Process node before children
Inorder	Left → Node → Right	Node between subtrees
Postorder	Left → Right → Node	Process node after children

A recursive traversal is short because each recursive call remembers where to return. The later lecture section compares this hidden call stack with an explicit stack used in iterative traversal.

8. Iterative preorder traversal using a stack

Preorder must print the current node before visiting its children. If we keep moving left, we would lose the addresses of right subtrees unless we save them. A stack stores those pending right-child references in last-in, first-out order.

```
void preorderIterative(Node root) {
if (root == null) return;

Stack<Node> stack = new Stack<>();
Node current = root;

while (current != null || !stack.isEmpty()) {
while (current != null) {
System.out.print(current.data + " ");

if (current.right != null) {
stack.push(current.right);
}
current = current.left;
}

if (!stack.isEmpty()) {
current = stack.pop();
}
}
}
```

Trace the logic

• Start at root 50: print 50, push its right child 90, then move left.

• At 30: print 30, push its right child 40, then move left.

• At 10: print 10, push its right child 20, then move left.

• When current becomes null, pop 20, then later pop 40, then 90 as the saved right branches are processed.

The exact traversal output depends on the tree shown in the lecture; the important rule is: print current, save the right child if present, and move left. When the left path ends, pop a saved right subtree.

9. Breadth-first search (BFS) / level-order traversal

BFS visits nodes level by level, from left to right in a typical binary tree. A queue is the natural structure because nodes are processed in first-in, first-out order.

```
void levelOrder(Node root) {
if (root == null) return;

Queue<Node> queue = new LinkedList<>();
queue.offer(root);

while (!queue.isEmpty()) {
Node current = queue.poll();
System.out.print(current.data + " ");

if (current.left != null) queue.offer(current.left);
if (current.right != null) queue.offer(current.right);
}
}
```

Queue dry run

• Initially queue contains [50]. Remove 50 and print it; add its left and right children.

• Next remove the first child, print it, and append its children to the queue.

• Continue until the queue is empty.

10. DFS vs BFS

Feature	DFS (Depth-First Search)	BFS (Breadth-First Search)
Strategy	Go deep along a branch, then backtrack	Visit level by level
Typical structure	Recursion or explicit stack	Queue
Tree traversal examples	Preorder, inorder, postorder are DFS patterns	Level-order traversal
Auxiliary space	O(h) recursion/stack for a tree of height h; worst O(n)	O(w), where w is maximum width; worst O(n)
Common use	Explore paths/subtrees; recursive tree operations	Level-order output; shortest path in an unweighted graph

DFS and BFS are general search strategies, not only tree methods. In graphs, mark visited vertices to avoid revisiting nodes and getting stuck in cycles.

11. Using parent pointers to move through a tree

The lecture introduces the idea of storing a parent reference in each node. Parent links can make it possible to move upward without a separate recursion stack, but the traversal algorithm must carefully decide whether it arrived from the parent, left child, or right child.

```
class Node {
int data;
Node left, right, parent;
}
```

• When attaching a child, set both directions consistently: parent.left = child and child.parent = parent.

• If parent pointers are not maintained correctly during insertion or deletion, upward traversal can become incorrect.

The transcript discusses this as a way to reason about non-recursive tree traversal. The exact final algorithm should be followed from the lecture's code and diagram; this note keeps the central concept without assuming a particular tree representation beyond parent links.

12. Deleting a particular node in a binary search tree

The lecture moves from clearing an entire tree to deleting an individual node. In a BST, the correct operation depends on whether the target is a leaf, has one child, or has two children. Parent links or a recursive return path are needed to reconnect the tree correctly.

The three structural cases

• Leaf node: remove the parent's reference to it.

• One child: connect the parent directly to the node's child.

• Two children: replace the node's value with its inorder successor (smallest value in its right subtree) or inorder predecessor (largest value in its left subtree), then remove that successor/predecessor from its original position.

The transcript includes implementation discussion and diagrams; use the exact lecture version if your instructor expects a specific implementation style. The cases above summarize the standard BST deletion logic to make the decision process clear.

BST deletion checklist

• Search for the target using BST comparisons.

• Determine how many children the target has.

• Reconnect the parent/child links without losing a subtree.

• If using parent pointers, update the parent reference of any moved child.

• After deletion, test inorder traversal; it should still be sorted for a valid BST.

13. Complexity quick reference

Operation	Typical time	Extra space
HashMap get/put	Average O(1), worst-case can degrade	Depends on buckets/entries
Tree height calculation	O(n)	O(h) recursion stack
Recursive traversal	O(n)	O(h) recursion stack
Iterative preorder	O(n)	O(h) saved stack in typical tree shape; worst O(n)
Level-order traversal	O(n)	O(w) queue; worst O(n)
BST search/insert/delete	O(h); balanced ~O(log n), skewed worst O(n)	O(1) iterative or O(h) recursive

14. Interview questions

Q. Why does HashMap need hashCode() and equals()?

Answer: hashCode() narrows the search to a bucket; equals() identifies the exact key among entries in that bucket.

Q. Can two unequal objects have the same hash code?

Answer: Yes. That is a collision.

Q. What must be true for equal objects?

Answer: Their hash codes must be equal.

Q. Why use postorder to clear a tree?

Answer: It processes child subtrees before the parent node.

Q. What is the height of an empty tree in this lecture's convention?

Answer: -1; therefore a leaf has height 0.

Q. Why is a stack useful for iterative preorder?

Answer: It saves right subtrees while traversal follows the left path.

Q. Why is a queue used for level-order traversal?

Answer: FIFO ordering naturally processes nodes level by level.

Q. What does a wrapper method achieve?

Answer: It hides internal node details and provides a readable public operation.

Q. What is the key invariant after BST deletion?

Answer: The BST ordering property must still hold.

15. Practice questions

1. Write pseudocode for open-addressing search that terminates when a key is absent.

2. Explain the difference between hashCode() and equals() using a collision example.

3. Why is it incorrect for equal Student objects to return different hash codes?

4. Write recursive Java code to calculate tree height using null height = -1.

5. Give preorder, inorder, postorder, and level-order sequences for a small tree of your choice.

6. Explain why iterative preorder pushes right children before moving left.

7. Write level-order traversal using Queue<Node>.

8. Describe the three cases in BST deletion.

9. After deleting a BST node, what property should inorder traversal demonstrate?

10. Compare DFS and BFS in terms of data structure and visitation order.

16. Short answer key

1. Follow the probe sequence; stop at an empty slot or after at most table.length probes.

2. hashCode selects a bucket; equals checks the actual key within that bucket.

3. The contract requires equal objects to have equal hash codes.

4. Base case returns -1; otherwise return max(height(left), height(right)) + 1.

5. Depends on the chosen tree; use the three traversal orders and queue-based level order.

6. Right-subtree references would otherwise be lost as the pointer moves down the left side; the stack preserves them in LIFO order.

7. Enqueue root; repeatedly dequeue, print, and enqueue non-null left then right children.

8. Leaf: unlink; one child: bypass target; two children: replace using successor/predecessor then remove that node.

9. Inorder output remains sorted.

10. DFS uses recursion/stack and explores depth-first; BFS uses a queue and explores level by level.

Final revision sheet

• Open addressing: follow the same probe sequence for search and insertion; include a bounded probe count.

• HashMap: key.hashCode() → bucket → equals() comparison.

• Equal objects must have equal hash codes; unequal objects may collide.

• Delete all tree nodes: postorder; then set root = null.

• Height convention in this lecture: null = -1, leaf = 0, height = max(left, right) + 1.

• Preorder: Node–Left–Right; iterative version saves right branches on a stack.

- Level order/BFS: queue, process node, enqueue left then right.
- BST deletion: handle 0, 1, or 2 children and preserve ordering.

End of Day 7 notes. Some code is explanatory sample code, not a verbatim transcription of the instructor's implementation.